🟢 Aula 4 — Eventos e Emissão de Dados (4h)

🎯 Objetivos da aula

- Compreender **eventos nativos** e **eventos customizados** no Vue.
- Utilizar corretamente o `$emit`.
- Implementar **comunicação filho → pai** (padrão real de mercado).
- Criar **componentes interativos** com atualização de estado centralizada.

---

1) Teoria (aprox. 1h30)

1.1 Eventos nativos

São eventos do próprio HTML, tratados pelo Vue com `v-on` ou `@`.

```
<button @click="salvar">Salvar</button> <input @input="onInput" />
```

✔️ O Vue apenas **escuta** eventos do DOM.

1.2 Eventos customizados

Eventos **criados pelo desenvolvedor**, usados para comunicação entre componentes.

- Não existem no DOM.
- São disparados com `$emit`.
- O componente pai escuta o evento.

```
this.$emit('meu-evento', payload)
```

No pai:

```
<MeuComponente @meu-evento="tratarEvento" />
```

1.3 `$emit` e boas práticas

- Nomear eventos em **kebab-case** (`salvar-usuario`).
- Emitir **dados mínimos necessários**.
- **Nunca alterar props no filho**.

1.4 Comunicação Filho → Pai (Fluxo)

```
Usuário → Evento (filho) → $emit → Pai recebe → Pai altera estado → Estado flui novamente via props
```

✔️ Fluxo previsível, fácil de manter e testar.

---

2) Prática (aprox. 2h30) — CRUD interativo com eventos

🎯 **Contexto**
Evoluir o cadastro de usuários:

- Botões **Ativar/Inativar**
- Botão **Remover**
- Toda alteração ocorre no **componente pai**
- Filhos apenas **emitem eventos**

2.1 Estrutura usada

```
src/
 ├─ components/
 │   ├─ UserForm.vue
 │   ├─ UserList.vue
 │   └─ UserItem.vue
 └─ App.vue
```

---

3) Código comentado — Comunicação filho → pai

3.1 `App.vue` — Pai (estado centralizado)

```
<template> <main class="container"> <h1>ILP3 – Vue.js Aula 4</h1> <UserForm @salvar="adicionarUsuario" /> <!-- Escuta eventos vindos da lista --> <UserList :usuarios="usuarios" @alternar-status="alternarStatus" @remover="removerUsuario" /> </main> </template> <script> import UserForm from './components/UserForm.vue' import UserList from './components/UserList.vue' export default { name: 'App', components: { UserForm, UserList }, data() { return { usuarios: [], proximoId: 1 } }, methods: { adicionarUsuario(dados) { this.usuarios.push({ id: this.proximoId++, ...dados }) }, alternarStatus(id) { const usuario = this.usuarios.find(u => u.id === id) if (usuario) { usuario.ativo = !usuario.ativo } }, removerUsuario(id) { this.usuarios = this.usuarios.filter(u => u.id !== id) } } } </script>
```

3.2 `UserForm.vue` — Evento customizado de criação

```
<template> <section class="card"> <h2>Novo Usuário</h2> <input v-model="nome" placeholder="Nome" /> <input v-model="email" placeholder="Email" /> <!-- Evento nativo do botão --> <button @click="salvar"> Salvar </button> </section> </template> <script> export default { name: 'UserForm', data() { return { nome: '', email: '' } }, methods: { salvar() { // Emissão de evento customizado this.$emit('salvar', { nome: this.nome, email: this.email, ativo: true }) this.nome = '' this.email = '' } } } </script>
```

3.3 `UserList.vue` — Repassando eventos

```
<template> <section class="card"> <h2>Usuários</h2> <p v-if="usuarios.length === 0"> Nenhum usuário cadastrado. </p> <ul v-else> <UserItem v-for="usuario in usuarios" :key="usuario.id" :usuario="usuario" <!-- Escuta eventos do item --> @alternar-status="$emit('alternar-status', usuario.id)" @remover="$emit('remover', usuario.id)" /> </ul> </section> </template> <script> import UserItem from './UserItem.vue' export default { name: 'UserList', components: { UserItem }, props: { usuarios: { type: Array, required: true } } } </script>
```

3.4 `UserItem.vue` — Componente interativo (emissor)

```
<template> <li class="item"> <strong>{{ usuario.nome }}</strong> — {{ usuario.email }} <span class="status" :class="{ ativo: usuario.ativo }" > {{ usuario.ativo ? 'Ativo' : 'Inativo' }} </span> <!-- Eventos customizados --> <button @click="$emit('alternar-status')"> {{ usuario.ativo ? 'Inativar' : 'Ativar' }} </button> <button @click="$emit('remover')"> Remover </button> </li> </template> <script> export default { name: 'UserItem', props: { usuario: { type: Object, required: true } } } </script> <style scoped> .item { margin-bottom: 10px; } .status { margin: 0 10px; padding: 3px 6px; border-radius: 6px; background: #ccc; } .status.ativo { background: #4caf50; color: white; } </style>
```

---

4) O que foi consolidado nesta aula

✔️ Eventos nativos

```
<button @click="salvar" />
```

✔️ Eventos customizados

```
this.$emit('alternar-status')
```

✔️ Comunicação Filho → Pai

- Filho **emite**
- Pai **decide e altera estado**
- Estado flui novamente via props

---

5) Atividade prática da aula

🎯 Desafio

1. Criar evento `editar` no `UserItem`
2. Emitir o usuário completo
3. No `App.vue`, capturar evento e:
    - Exibir dados no formulário
    - Atualizar usuário ao salvar

💡 Dica:

```
this.$emit('editar', this.usuario)
```

---

6) Checklist de encerramento

- Usou `$emit`
- Separou responsabilidade entre pai e filho
- Criou componentes interativos
- Atualizou estado via eventos
- Entendeu fluxo unidirecional